NxtDevs — Complete System Documentation

> A comprehensive AI-powered Algorithmic Thinking Trainer that profiles *how* users think, identifies cognitive biases, and provides personalized coaching through adaptive practice, real-time duels, and conversational AI tutoring.

🏗️ Project Overview

NxtDevs is an **Algorithmic Thinking Trainer** that goes beyond traditional coding practice. Instead of just testing syntax, it profiles *how* a user thinks about problems, identifies cognitive biases (e.g., Greedy Bias, Constraint Blindness), and provides personalized coaching.

Core Value Proposition

1. **Thinking Profiles**: Track 20+ cognitive axes to understand *why* users fail.
2. **Ranked 1v1 Duels**: Real-time competitive matches with ELO rating.
3. **AI Coaching**: Personalized explanations powered by Gemini/Groq.
4. **Adaptive Questions**: Infinite practice via AI-generated variants.
5. **TutorAI Chat**: Conversational AI tutor with memory and web search capabilities.
6. **Analytics Reports**: AI-powered performance reports with PDF export.

🛠️ Tech Stack

Backend

Technology	Purpose
Python 3.11	Core language
FastAPI	REST API framework

Technology	Purpose
SQLModel	ORM (SQLAlchemy + Pydantic)
PostgreSQL	Production database (Supabase)
WebSockets	Real-time duel communication
Redis	Caching, chat history, Celery broker
Celery	Background task processing
Google Gemini 2.5	Primary LLM for AI features
Groq (Llama 3.3)	Fallback LLM
LangChain	AI orchestration & memory

Frontend

Technology	Purpose
Next.js 16	React framework (App Router)
TypeScript	Type safety
Tailwind CSS	Styling
Framer Motion	Animations
Lucide React	Icons
Recharts	ELO graphs & analytics charts

Infrastructure

Technology	Purpose
Supabase	Managed PostgreSQL database
Redis	Session storage, Celery broker/backend
Google OAuth	Social authentication
Wikipedia/ArXiv API	TutorAI knowledge retrieval

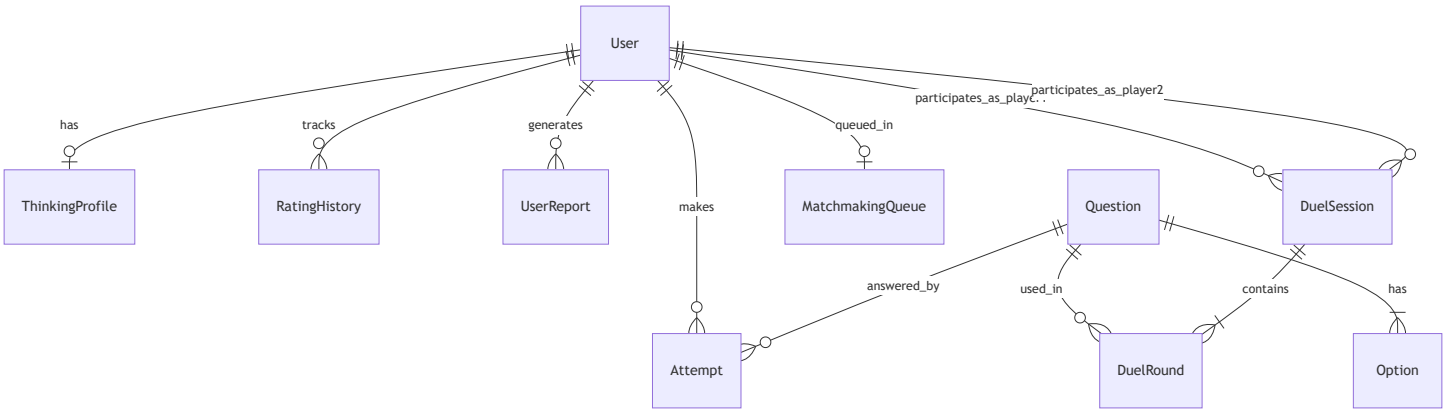
📂 Project Structure

```
Axiom/
├── backend/
│   ├── api/                    # FastAPI Routers
│   │   ├── auth.py             # Login, Register, Google OAuth
│   │   ├── questions.py        # Practice, Submit, Profile, Leaderboard
│   │   ├── duels.py            # 1v1 Real-time WebSocket API
│   │   ├── tutor.py            # TutorAI Chat endpoints
│   │   ├── reports.py          # Analytics & PDF generation
│   │   └── contracts.py        # Pydantic Request/Response Models
│   ├── core/
│   │   ├── db.py               # PostgreSQL/Supabase connection
│   │   └── auth.py             # JWT & OAuth logic
│   ├── engine/
│   │   ├── scoring.py          # ELO Rating Calculation
│   │   └── orchestrator.py     # Question Selection Logic
│   ├── models/                 # SQLModel ORM Definitions
│   │   ├── user_state.py       # User, ThinkingProfile, Attempt
│   │   ├── canonical.py        # Question, Option, ThinkingAxis
│   │   ├── duel_models.py      # DuelSession, DuelRound, MatchmakingQueue
│   │   └── report_models.py    # UserReport model
│   ├── services/               # Business Logic
│   │   ├── ai_service.py       # Gemini/Groq Integration
│   │   ├── tutor_service.py    # TutorAI with LangChain + Redis memory
│   │   ├── matchmaking.py      # ELO-based Player Pairing
│   │   └── celery_tasks.py     # Background report generation
│   ├── prompts.yaml            # AI prompt templates
│   └── main.py                 # FastAPI App Entry Point
├── frontend/
│   ├── app/                    # Next.js 16 App Router
│   │   ├── (landing)/          # Public Landing Page
│   │   ├── dashboard/          # Main User Dashboard
│   │   ├── practice/           # Solo Question Practice
│   │   ├── rating/             # Rated Practice Mode
│   │   ├── duel/               # 1v1 Duel Interface
│   │   ├── chat/               # TutorAI Chat Interface
│   │   ├── reports/            # Analytics & Reports
│   │   ├── leaderboard/        # Global Rankings
│   │   ├── profile/            # User Stats & History
│   │   └── settings/           # Account Settings
│   ├── components/
```

```
│   │   ├── chat/                # TutorAI components
│   │   │   ├── ChatInterface.tsx
│   │   │   ├── MessageBubble.tsx
│   │   │   ├── ContextSidebar.tsx
│   │   │   └── SuggestionChips.tsx
│   │   ├── landing/             # Hero, FeatureGrid, Footer
│   │   ├── dashboard/           # QuestionCard, ProgressChart
│   │   ├── question/            # Question rendering components
│   │   ├── layout/              # Sidebar, TopNav, AppLayout
│   │   └── ui/                  # Reusable UI Components
│   └── lib/
│       ├── auth.ts              # Token Storage, Logout, API Wrapper
│       └── utils.ts             # Tailwind Merge (cn)
├── celery_app.py                # Celery worker configuration
└── celeryconfig.py              # Celery settings
```

🗄️ Database Schema

Entity Relationship Diagram



Tables

`user` (User Identity)

Column	Type	Description
`id`	UUID	Primary Key
`username`	String	Unique, indexed

Column	Type	Description
`email`	String	Optional
`password_hash`	String	BCrypt hash (null for OAuth users)
`created_at`	DateTime	Account creation timestamp

`thinkingprofile` (Cognitive Data)

Column	Type	Description
`id`	UUID	Primary Key
`user_id`	UUID	Foreign Key → User
`elo_rating`	Float	Global ELO (default: 1200)
`volatility`	Float	Glicko-style uncertainty
`greedy_bias`	Float	0.0-1.0 tendency score
`constraint_blindness`	Float	0.0-1.0 tendency score
`premature_optimization`	Float	0.0-1.0 tendency score
`axis_performances`	JSON	`{axis: {elo, confidence}}`
`updated_at`	DateTime	Last profile update

`question` (Content)

Column	Type	Description
`id`	UUID	Primary Key
`content`	Text	Full question markdown
`question_type`	String	e.g., `constraint_selection`
`difficulty_tier`	Integer	1-5 scale
`tags`	JSON	`["dp", "greedy"]`
`thinking_axis`	String	ThinkingAxis enum value
`variant_group_id`	UUID	Links related variants

Column	Type	Description
`is_canonical`	Boolean	True = used in Rating Mode
`explanation`	Text	Static explanation (legacy)

`option` (Answer Choices)

Column	Type	Description
`id`	UUID	Primary Key
`question_id`	UUID	Foreign Key → Question
`content`	Text	Option text
`is_correct`	Boolean	Correct answer flag
`approach_type`	String	e.g., `Optimal` , `Greedy Trap`
`assumptions_broken`	JSON	What traps this exploits

`attempt` (User Submissions)

Column	Type	Description
`id`	UUID	Primary Key
`user_id`	UUID	Foreign Key → User
`question_id`	UUID	Answered question
`selected_option_id`	UUID	User's choice
`is_correct`	Boolean	Result
`time_taken_ms`	Integer	Response time
`mode`	String	`practice` or `rating`
`rating_before`	Float	ELO before attempt
`rating_after`	Float	ELO after attempt
`timestamp`	DateTime	Submission time

`ratinghistory` (ELO Graph Data)

Column	Type	Description
`id`	UUID	Primary Key
`user_id`	UUID	Foreign Key → User
`rating`	Float	Rating at this point
`delta`	Float	Change from previous
`attempt_id`	UUID	Triggering attempt
`timestamp`	DateTime	Event time

`duelsession` (1v1 Matches)

Column	Type	Description
`id`	UUID	Primary Key
`player1_id`	UUID	FK → User
`player2_id`	UUID	FK → User
`player1_rating_start`	Float	ELO at match start
`player2_rating_start`	Float	ELO at match start
`player1_score`	Integer	Correct answers
`player2_score`	Integer	Correct answers
`current_round`	Integer	0-4 (5 rounds total)
`status`	Enum	WAITING/COUNTDOWN/IN_PROGRESS/COMPLETED
`winner_id`	UUID	Match winner
`player1_rating_delta`	Float	ELO change
`player2_rating_delta`	Float	ELO change

`duelround` (Individual Questions in Duel)

Column	Type	Description
`id`	UUID	Primary Key
`duel_id`	UUID	FK → DuelSession
`round_number`	Integer	0-4
`question_id`	UUID	FK → Question
`player1_option_id`	UUID	P1's answer
`player1_is_correct`	Boolean	P1 result
`player1_time_ms`	Integer	P1 response time
`player2_option_id`	UUID	P2's answer
`player2_is_correct`	Boolean	P2 result
`player2_time_ms`	Integer	P2 response time

`matchmakingqueue` (Waiting Players)

Column	Type	Description
`id`	UUID	Primary Key
`user_id`	UUID	FK → User (unique)
`elo_rating`	Float	Rating snapshot
`joined_at`	DateTime	Queue entry time
`expanded_range`	Boolean	True after 30s timeout

`userreport` (Performance Reports)

Column	Type	Description
`id`	UUID	Primary Key
`user_id`	UUID	FK → User
`status`	Enum	PENDING/GENERATING/GENERATED/FAILED

Column	Type	Description
`report_type`	String	Type of report
`report_data`	JSON	Generated report content
`error_message`	String	Error details if failed
`created_at`	DateTime	Creation timestamp
`updated_at`	DateTime	Last update

🔌 API Endpoints

Authentication (`/auth`)

Method	Endpoint	Description
POST	`/auth/register`	Create account (username/email/password)
POST	`/auth/login`	Login with credentials → token
GET	`/auth/me`	Get current user from token
GET	`/auth/google/login`	Get Google OAuth URL
POST	`/auth/google/callback`	Exchange code for token
POST	`/auth/logout`	Invalidate token

Questions (`/api/v1`)

Method	Endpoint	Description
GET	`/next`	Get next question (uses Orchestrator)
POST	`/submit/{question_id}`	Submit answer, update profile
GET	`/profile`	Get user's thinking profile & stats
PUT	`/profile`	Update username/email

Method	Endpoint	Description
GET	`/leaderboard`	Global ELO rankings (cached)
GET	`/rating-history`	User's ELO graph data

Duels (`/duels`)

Method	Endpoint	Description
POST	`/queue/join`	Join matchmaking queue
POST	`/queue/leave`	Leave matchmaking queue
GET	`/{duel_id}/state`	Get current duel state
POST	`/{duel_id}/submit`	Submit answer for current round
WS	`/{duel_id}/ws`	Real-time WebSocket connection

TutorAI (`/tutor`)

Method	Endpoint	Description
POST	`/chat`	Chat with AI tutor
POST	`/chat/stream`	Stream chat responses
POST	`/clear-history`	Clear conversation history
GET	`/state`	Get student stats for tutor context
GET	`/suggestions`	Get contextual chat suggestions

Reports (`/reports`)

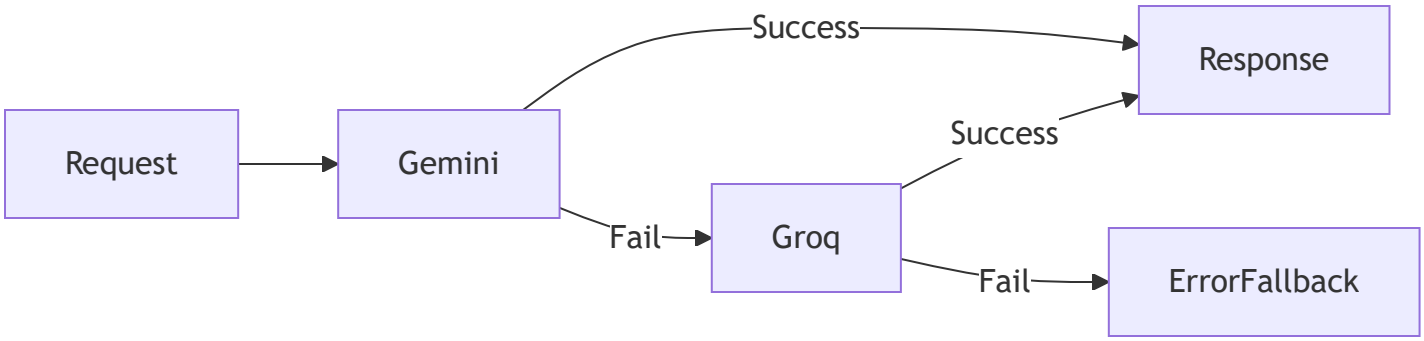
Method	Endpoint	Description
POST	`/generate`	Start async report generation
GET	`/`	List all user reports
GET	`/{report_id}`	Get specific report details

Method	Endpoint	Description
GET	`/{report_id}/download`	Download report as PDF

⚙️ Core Services

1. AIService (`ai_service.py`)

Purpose: LLM-powered features with fallback chain.

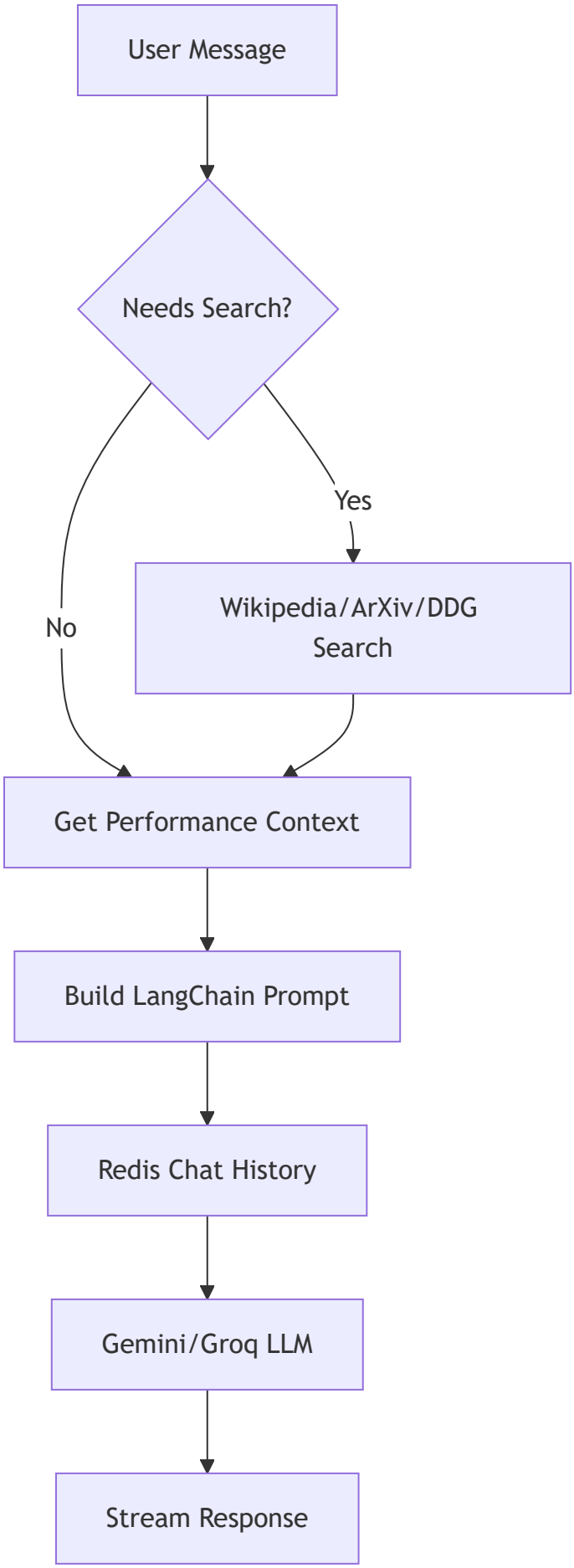


Methods:

- `generate_content(prompt)` : Generic LLM call
- `classify_question_axis(text)` : Auto-tag questions with ThinkingAxis
- `generate_personalized_explanation(profile, question, answer)` : Tailored feedback

2. TutorAIService (`tutor_service.py`)

Purpose: Conversational AI tutor with memory and knowledge retrieval.

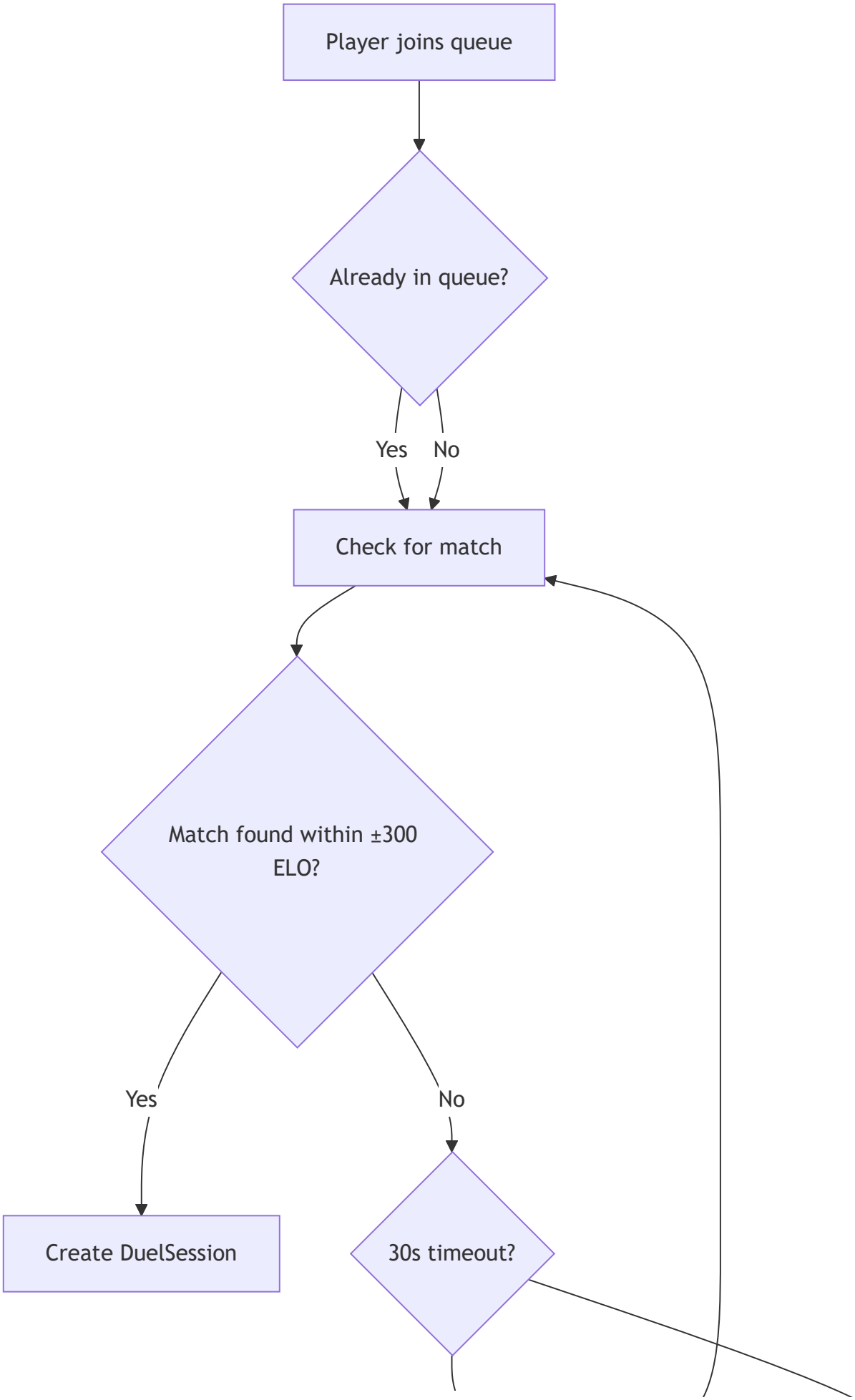

User Message
Needs Search?
Yes
No
Wikipedia/ArXiv/DDG Search
Get Performance Context
Build LangChain Prompt
Redis Chat History
Gemini/Groq LLM
Stream Response

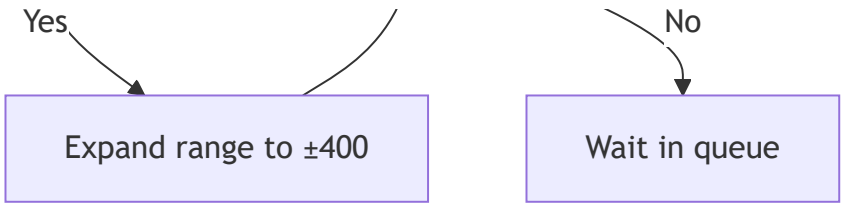
Features:

- **Redis Chat Memory**: Persistent conversation history per user
- **Performance Context**: Injects user's quiz stats into prompts
- **Multi-Source Search**: Wikipedia → ArXiv → DuckDuckGo fallback
- **Streaming Responses**: Real-time token streaming
- **Configurable Prompts**: YAML-based prompt templates

3. MatchmakingService (`matchmaking.py`)

Purpose: ELO-based player pairing for duels.


Player joins queue
Already in queue?
Yes
No
Check for match
Match found within ±300 ELO?
Yes
No
Create DuelSession
30s timeout?



4. ScoringEngine (`scoring.py`)

Purpose: ELO rating calculations.

Formula (Standard ELO):

```
Expected Score = 1 / (1 + 10^((opponent_rating - player_rating) / 400))
New Rating = Old Rating + K * (Actual Score - Expected Score)
```

- K-factor: 32 (standard)
- Win = 1.0, Loss = 0.0, Draw = 0.5

5. Celery Tasks (`celery_tasks.py`)

Purpose: Background processing for compute-intensive operations.

Tasks:

- `generate_report_task` : Async report generation with AI insights
- `generate_questions_batch_task` : Batch question generation via AI

Configuration:

- Redis as broker and result backend
- Task acknowledgment after completion
- Automatic retry with exponential backoff

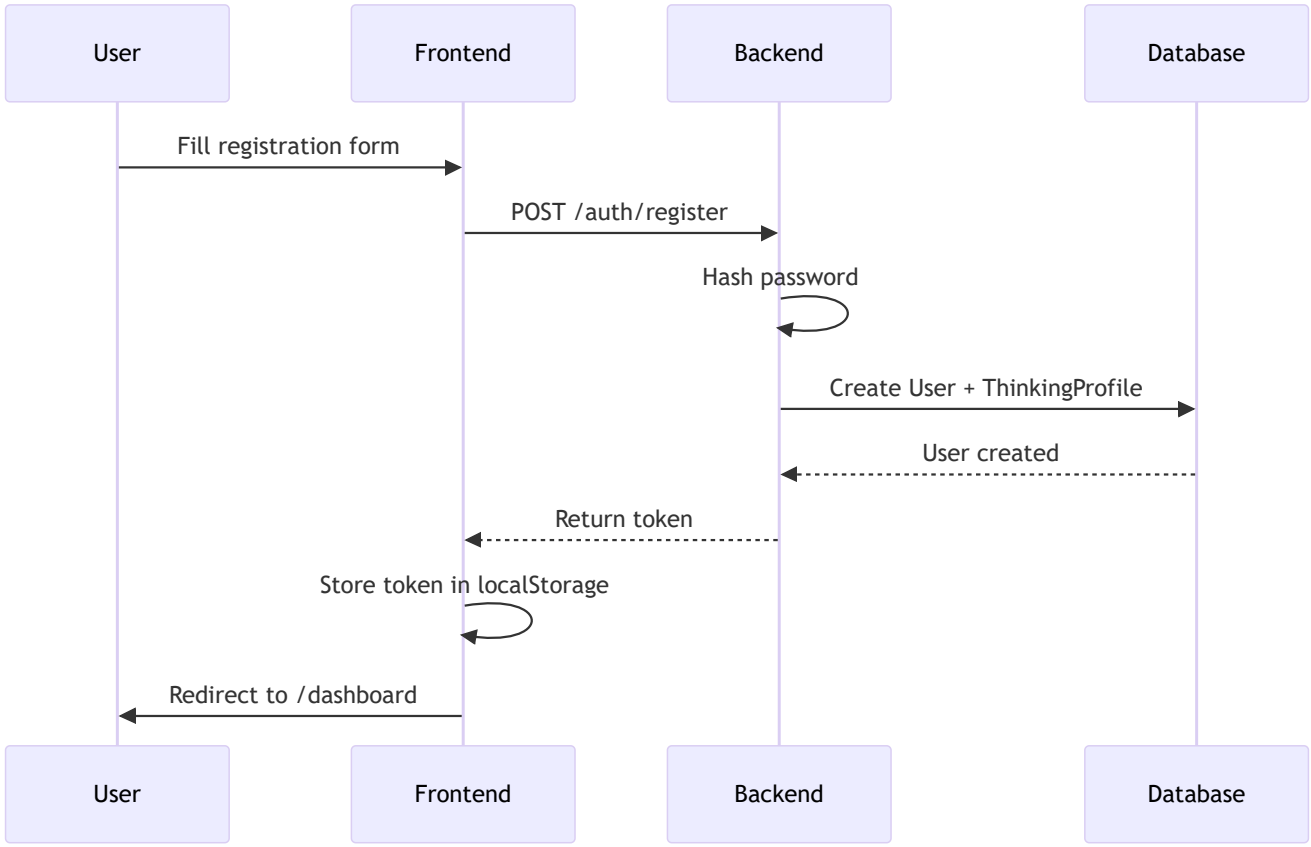
🎨 Frontend Pages

Route	Component	Description
`/`	LandingPage	Marketing page with Hero, Features
`/login`	LoginPage	Glassmorphism auth overlay

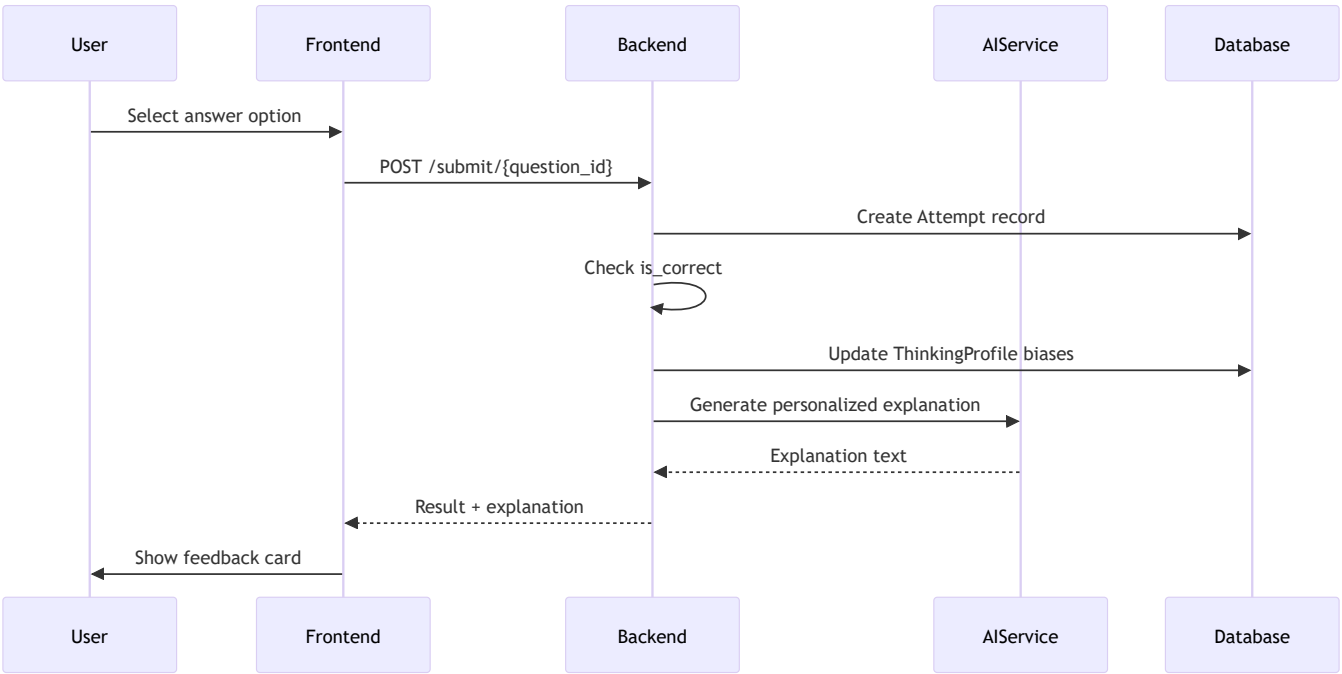
Route	Component	Description
`/register`	RegisterPage	Account creation
`/dashboard`	DashboardPage	Main hub, recent attempts
`/practice`	PracticePage	Solo mode, any question
`/rating`	RatingPage	Ranked mode, canonical only
`/duel`	DuelPage	1v1 real-time interface
`/chat`	ChatPage	TutorAI conversational interface
`/reports`	ReportsPage	Analytics with charts & PDF export
`/leaderboard`	LeaderboardPage	Global ELO rankings
`/profile`	ProfilePage	Stats, ELO graph, duel history
`/settings`	SettingsPage	Account preferences

🔄 System Flowcharts

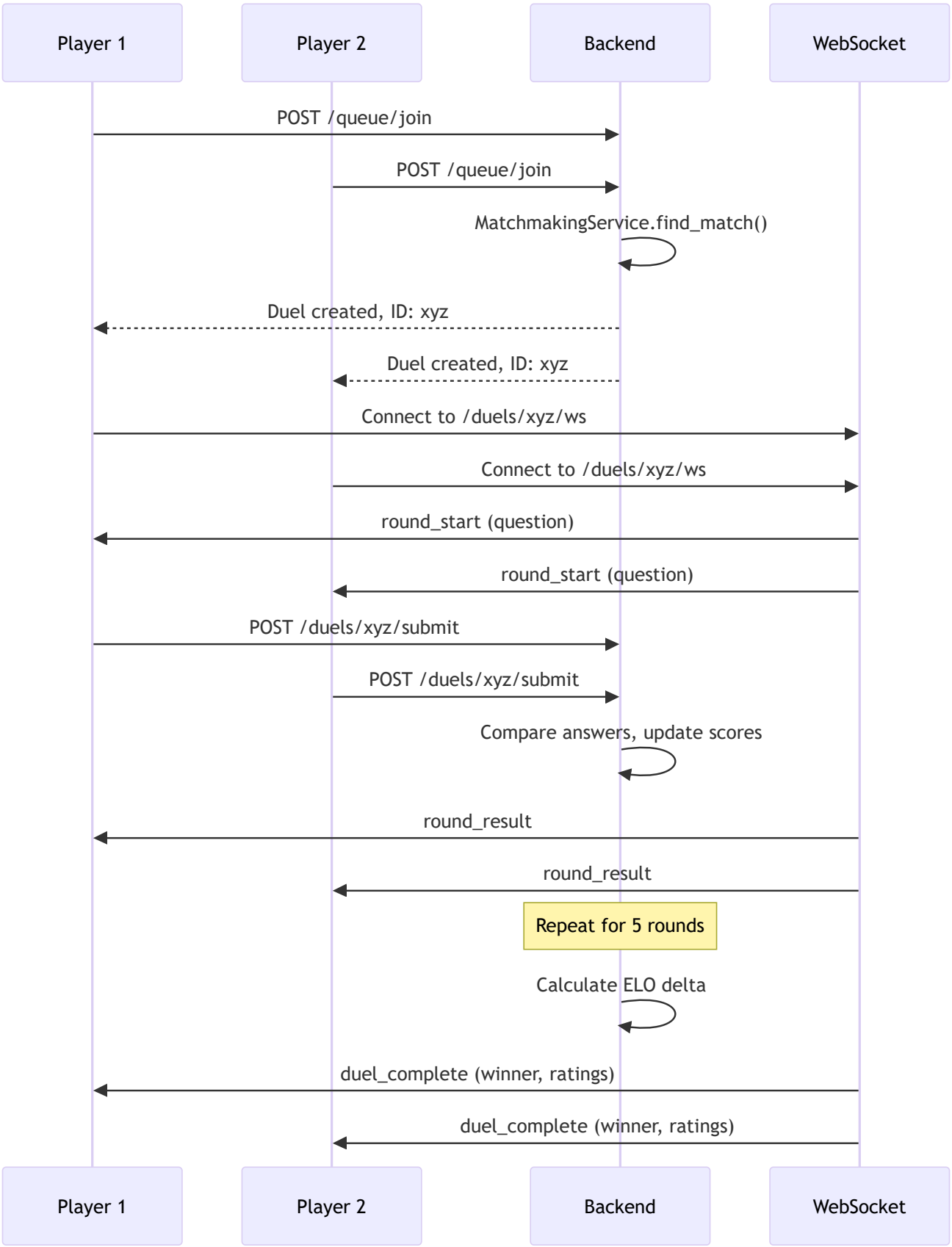
User Registration & Login



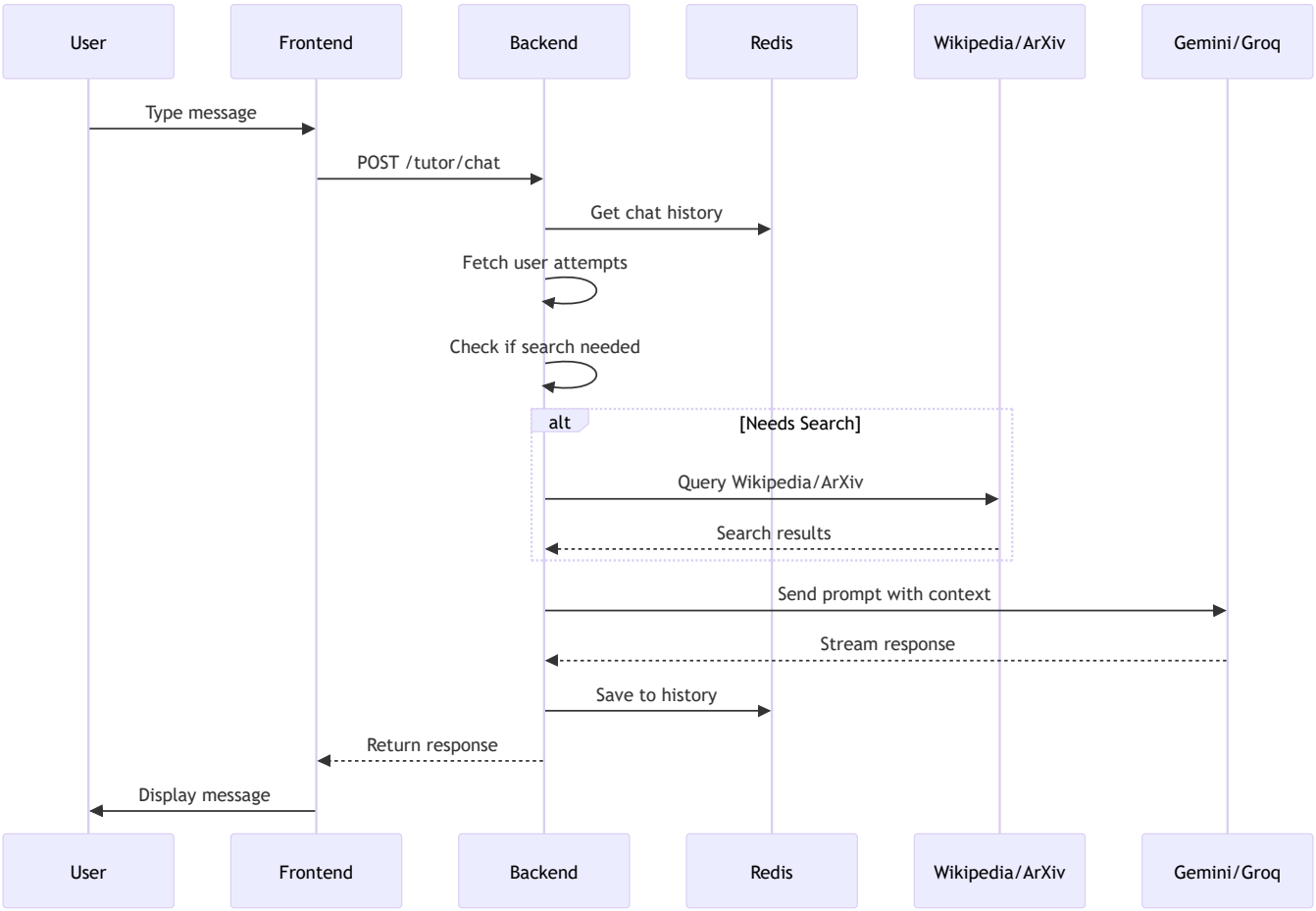
Question Submission (Practice Mode)



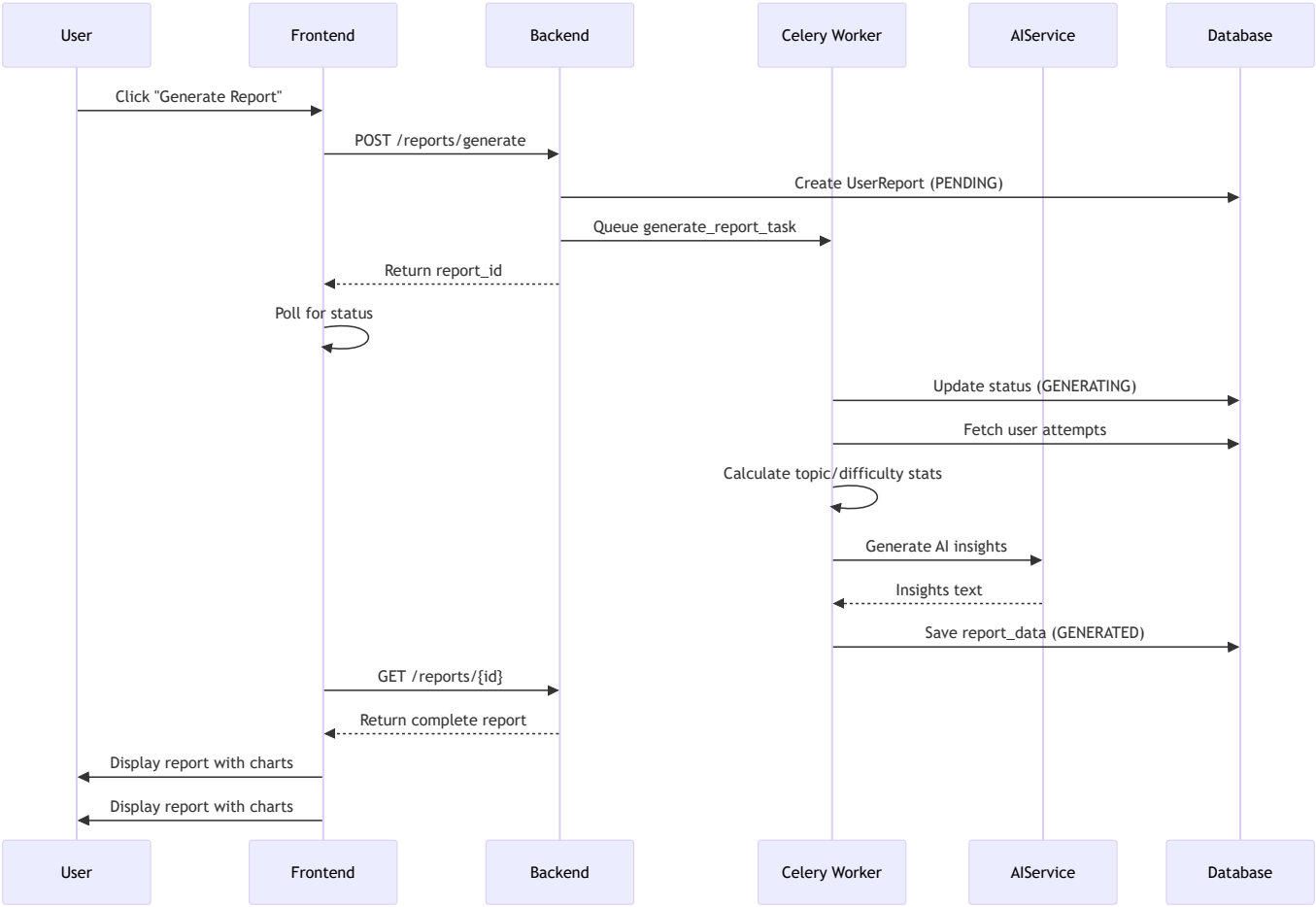
1v1 Duel Flow



TutorAI Chat Flow



Report Generation Flow



🚀 Implemented Features Summary

✅ Phase 1: Core Question Engine

- ☑ Question & Option models
- ☑ Practice mode (`GET /next` , `POST /submit`)
- ☑ Basic profile tracking

✅ Phase 2: Rating System

- ☑ ELO rating calculation
- ☑ ThinkingProfile with bias scores
- ☑ `is_canonical` field for Rating Mode integrity
- ☑ Leaderboard with Redis caching (30s TTL)

✅ Phase 3: 1v1 Duels

- [x] Matchmaking queue with ELO range
- [x] WebSocket real-time communication
- [x] 5-round duel sessions
- [x] Score tracking & winner determination
- [x] Post-duel ELO updates

✅ Phase 4: AI Integration

- [x] Gemini + Groq fallback chain
- [x] Auto-tagging questions with ThinkingAxis
- [x] Personalized explanations based on user profile

✅ Phase 5: Authentication

- [x] Username/Password registration & login
- [x] Google OAuth integration
- [x] Token-based session management
- [x] Sidebar logout button

✅ Phase 6: TutorAI Chat

- [x] LangChain-powered conversational AI
- [x] Redis-backed chat history persistence
- [x] Performance context injection
- [x] Wikipedia/ArXiv/DuckDuckGo search integration
- [x] Streaming response support
- [x] Contextual chat suggestions
- [x] Student stats dashboard

✅ Phase 7: Analytics & Reports

- [x] Async report generation via Celery
- [x] AI-powered performance insights
- [x] Topic and difficulty breakdown
- [x] Accuracy trend charts
- [x] PDF export with Matplotlib charts
- [x] Interactive report viewer

✅ Phase 8: Landing Page & UI

- [x] Hero section with CTAs
- [x] Infinite Logo Loop (tech stack)
- [x] Interactive Challenge Teaser
- [x] Visual Pipeline ("How It Works")
- [x] Custom Features Section (Aceternity-style)
- [x] Premium dark theme UI
- [x] Framer Motion animations throughout

🔮 Roadmap (Coming Soon)

🎯 LeetCode Integration

- [ ] Username linking for auto-sync
- [ ] Periodic profile analysis
- [ ] Problem-solving pattern detection
- [ ] Integration with Thinking Profiles

👩‍💻 Code Review

- [ ] Paste code for AI review
- [ ] Pattern and anti-pattern detection
- [ ] Interview readiness scoring

🎤 AI Interview (Major Feature)

- [ ] Live coding interview simulation
- [ ] Voice/video interaction
- [ ] Real-time behavioral analysis
- [ ] Comprehensive feedback reports

🔐 Environment Variables

Backend (`.env`)

```
# Database (Supabase PostgreSQL)
DATABASE_URL=postgresql://postgres:password@db.xxx.supabase.co:5432/postgres

# AI Services
GEMINI_API_KEY=your_gemini_key
GEMINI_MODEL=gemini-2.5-flash
GROQ_API_KEY=your_groq_key
GROQ_MODEL=llama-3.3-70b-versatile

# Redis
REDIS_URL=redis://localhost:6379/0
REDIS_HOST=localhost
REDIS_PORT=6379

# Celery
CELERY_BROKER_URL=redis://localhost:6379/0
CELERY_RESULT_BACKEND=redis://localhost:6379/0

# Google OAuth
GOOGLE_CLIENT_ID=your_client_id
GOOGLE_CLIENT_SECRET=your_client_secret
GOOGLE_REDIRECT_URI=http://localhost:3000/login
```

Frontend

- Uses environment-based API URL (defaults to `http://localhost:8000`)

🛠️ Running the Project

Backend

```
cd backend
python -m venv venv
.\venv\Scripts\activate  # Windows
pip install -r requirements.txt
uvicorn backend.main:app --reload --port 8000
```

Celery Worker

```
# From project root
celery -A celery_app worker --loglevel=info --pool=solo
```

Frontend

```
cd frontend
npm install
npm run dev
```

Redis (Required)

```
# Windows: Use Docker or WSL
docker run -d -p 6379:6379 redis:alpine

# Or via WSL
sudo service redis-server start
```

Last Updated: 2026-01-19